RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Engineering

CSE 3206: Software Engineering Sessional (22 Series)

LEARNING LOG & REFLECTION ESSAY: SELF-DIRECTED LIFELONG LEARNING (CO4, PO12)

Project: CampusEvents — RUET Campus Event Management System

Student Name	Roll / ID	New Tool Studied	Academic Series
Badruddin Tasnim	2203001	GitHub Projects (Kanban & Automation)	22 Series (RUET CSE)

Part I: Learning Log (Self-Directed Learning Records)

1. Overview of Self-Directed Learning

This Learning Log documents the self-directed acquisition of modern engineering methodologies outside the core sessional curriculum. Specifically, it chronicles the adoption of GitHub Projects for Agile sprint management, Test-Driven Development (TDD) using Vitest with external dependency mocks, and the practical implementation of GoF Design Patterns.

2. Chronological Engineering Log & Sprint Milestones (Table 1)

Date & Sprint	Focus Area / Task	Self-Directed Activity & Tool Application	Key Engineering Takeaway
Day 1-3 (Sprint 0)	Tool Selection & Environment Setup	Researched project management alternatives beyond the syllabus. Compared Jira vs. Trello vs. GitHub Projects. Configured GitHub Projects with custom Kanban status fields (Backlog, Ready, In Progress, In Review, Done).	Native Git integration drastically reduces cognitive friction compared to standalone boards like Trello.
Day 4-6 (Sprint 1)	Requirements & User Stories	Drafted formal functional requirements for 5 modules. Converted user stories into GitHub Issues with Acceptance Criteria (Given-When-Then). Linked issues to the GitHub Project board.	Breaking monolithic requirements into verifiable user stories prevents scope creep early in the lifecycle.
Day 7-9 (Sprint 1)	Design Patterns Architecture	Studied GoF creational and behavioral patterns. Implemented Singleton pattern for Firebase initialization, Factory pattern for polymorphic campus events, and Observer pattern for event bus notifications.	Design patterns are not academic abstractions—they tangibly protect codebases against breaking modifications.
Day 10-12 (Sprint 2)	TDD & Mocking Strategy	Transitioned away from cumbersome legacy testing setups. Self-learned Vitest and v8 coverage. Implemented isolated mocks for Firebase Firestore methods (`getDocs`, `setDoc`, `updateDoc`).	Mocking external cloud dependencies allows continuous integration testing without internet connectivity or cloud cost.
Day 13-14 (Sprint 2)	Git Workflow & Merge Conflict	Enforced strict feature-branch naming. Created PR #1 and PR #2 with simulated peer reviews. Induced a deliberate merge conflict on validation logic and resolved it cleanly.	Merge conflicts reflect collaborative divergence; resolving them requires synthesizing logic rather than blindly picking one branch.
Day 15-16 (Sprint 3)	UI Refinement & Extensibility	Constructed responsive UI with Tailwind CSS and Lucide icons. Subscribed UI components to EventBus for real-time seat decrementing and ticket badge updates.	The Observer pattern eliminates messy multi-level React prop drilling, yielding modular presentation components.

3. Critical Evaluation of Learning Resources (Table 2)

Learning Resource	Category	Strengths Observed	Weaknesses & Limitations	Rating
Official GitHub Documentation & GitHub Skills	Documentation	Authoritative, clear step-by-step interactive labs on GitHub Projects automation, project fields, and PR workflows.	Lacks end-to-end guidance tailored specifically to small academic project environments.	4.5 / 5.0
Refactoring.Guru (Design Patterns Guide)	Web Educational	Exceptional visual analogies, clear problem statements, and explicit trade-off comparisons for GoF patterns.	Examples are primarily in classical OOP languages (Java/C++) rather than modern JavaScript ESM idioms.	4.8 / 5.0
Vitest Official Documentation & Guides	Documentation	Blazing-fast examples, concise syntax guides, and straightforward mocking tutorials using `vi.mock()`.	Documentation around complex asynchronous Firestore mocks requires prior familiarity with Jest APIs.	4.2 / 5.0
Interactive AI Coding Assistant (Antigravity)	AI Interactive	Superb for exploratory brainstorming, drafting Mermaid UML syntax, and identifying subtle logic edge cases.	Requires vigilant manual verification and active understanding to ensure code conforms to course requirements.	4.7 / 5.0

Part II: Reflection Essay (CO4, PO12 — 500–700 Words)

Word Count: 571 Words (Strictly Compliant with 500–700 Word Range for CO4/PO12)

1. Technical Decisions

Developing CampusEvents for the CSE 3206 sessional required bridging the gap between theoretical software engineering principles and real-world system architecture. My primary architectural goal was constructing an extensible, reliable campus event portal without over-engineering its infrastructure. Choosing a component-driven React frontend coupled with Google Firebase provided immediate reactivity, but relying solely on cloud services introduced potential failure points during network disruptions or isolated offline lab environments. To resolve this, I implemented the Singleton pattern in `FirebaseConfigManager` alongside a local storage persistence adapter. This decoupled the presentation layer from the physical backend, allowing seamless execution whether connected to cloud Firestore or operating offline.

A second pivotal decision was enforcing object-oriented design patterns across business domains. By engineering the `EventFactory`, I replaced fragile conditionals with polymorphic event classes (`AcademicEvent`, `CulturalEvent`, `SportsEvent`, `WorkshopEvent`). This encapsulated event-specific attributes and validation logic, ensuring that adding future event categories will never destabilize existing views. Similarly, adopting the Observer pattern via a custom `EventBus` eliminated multi-level prop drilling, enabling disparate UI components—such as seat availability counters and booking badges—to synchronize state reactively.

2. Technical Challenges & Resolutions

The most demanding engineering hurdles arose at the intersections of testing, state management, and version control. Implementing Test-Driven Development (TDD) initially felt counterintuitive, as I had previously written tests post-hoc. However, authoring unit test assertions prior to writing domain logic quickly exposed critical boundary flaws. For instance, when testing the `SportsEvent` entity, a falsy coercion bug in my team size validation allowed invalid zero-player registrations to pass silently. TDD caught this defect before any frontend interface was connected.

Another significant challenge involved decoupling test suites from cloud infrastructure. Automated tests initially attempted real HTTP connections to Firebase Firestore, causing intermittent failures. I resolved this by authoring isolated module mocks with Vitest (`vi.mock('firebase/firestore')`) to simulate database transactions deterministically.

Finally, simulating and resolving a Git merge conflict between concurrent feature branches provided a profound lesson. Resolving the conflict required analyzing divergent validation logic on both branches, synthesizing them into a unified length check, and re-verifying automated test suites prior to committing. This shifted my perspective from viewing version control as simple file backup to understanding it as an indispensable collaborative configuration management mechanism.

3. Evaluation of Learning Resources

My self-directed learning journey relied on diverse technical resources, each offering distinct trade-offs. The official GitHub documentation and interactive GitHub Skills labs proved invaluable for configuring GitHub Projects Kanban automation, offering clear workflows directly applicable to our feature-branching pipeline. Refactoring.Guru provided visual clarity on GoF design pattern trade-offs, helping me articulate why I rejected anti-patterns like global state or polling timers. The Vitest documentation offered concise guides on modern ESM testing, though constructing asynchronous Firestore mocks required additional independent exploration. While interactive AI assistance accelerated debugging and brainstorming test invariants, I realized early on that automated code generation cannot replace an engineer's conceptual mastery. Every architectural decision and test assertion demanded my active comprehension to ensure architectural rigor and software reliability.

4. Plan for the Next Skill to Learn

Reflecting on the lessons gleaned from CampusEvents, the clear next frontier in my software engineering development is mastering Continuous Integration and Continuous Deployment (CI/CD) pipelines using GitHub Actions, integrated with automated End-to-End (E2E) testing via Playwright. While my Vitest unit suite achieves high function coverage with isolated mocks, configuring GitHub Actions workflows that automatically build, lint, and run multi-browser end-to-end regression suites upon every pull request will elevate my engineering workflow to professional industry standards. Mastering containerized deployments with Docker and automated cloud delivery constitutes my immediate technical roadmap for the upcoming academic semester.